be left to manual testing. Others may only be of marginal interest to the business, and might be better off implemented as unit or integration tests. Still others may be experimental, and might not be understood well enough to define clear scenarios; in this case, it could be worthwhile to do some initial prototyping to get a better feel for what's really needed.[1]

But when a scenario can be automated, when it makes sense to do so, and when it's done well, automating the scenario brings its own set of undeniable benefits:

- *Testers spend less time on repetitive regression testing.* When acceptance criteria and the corresponding scenarios are written in close collaboration with testers, automated versions of these scenarios give testers more confidence in new releases. The testers can understand and relate more easily to what the automated tests are verifying, because they took part in defining them. In addition, the application that the testers receive for testing will have already passed a broad range of simpler test cases, letting the testers focus on more complex or exploratory testing.
- *New versions can be released faster and more reliably.* Because less manual testing is required, new releases can be pushed out more efficiently. New versions are less likely to introduce regressions. Comprehensive automated testing is essential if you're trying to implement continuous integration, continuous delivery, or continuous deployment (see chapter 12).
- *The automated scenarios give a more accurate vision of the current state of the project.* You can use them to build a progress dashboard that describes which features have been delivered and how they've been tested, based on the results of the automated scenarios.

Continuous integration, continuous delivery, and continuous deployment

Continuous integration is a practice that involves automatically building and testing a project whenever a new code change is committed to the source code repository. Continuous integration is a valuable feedback mechanism, alerting developers to potential integration issues or regressions as early as possible. But to be really effective, continuous integration relies strongly on a robust and comprehensive set of automated tests.

Continuous delivery is an extension of continuous integration, where every build is a potential release. Whenever a developer puts new code into the source code repository, a build server compiles a new release candidate version. If this release candidate passes a series of automated quality checks (unit tests, automated acceptance tests, automated performance tests, code quality metrics, and so on), it can be pushed into production as soon as business stakeholders give their go-ahead.

[1] Highly experimental startup applications might fall into this category. If the business needs to get market feedback to discover what features they really need, it will be hard to formalize scenarios with very much detail.